COMP4423 – Assignment 2

Sunghak Heo (21097305D)

1. Introduction

Plant classification in real-world, which have great variability, is a common but challenging computer vision problem. This project aims to develop a robust plant image classification program that can distinguish multiple plant species collected from PolyU campus solely with traditional computer vision techniques and machine learning models.

In contrast to clean and well-organized datasets that are widely published throughout the websites like Kaggle and ImageNet, real-world plant images contain significant factors to consider. In particular, they exhibit a dynamic range of lighting, angle, environmental noise, and biological shape among intra-class. These variations lower the model performance.

1.1 Challenges in Campus Plant Classification

The primary challenges addressed in this project can be summarized as follows:

Different Lighting Condition
Plant images were taken under various lighting conditions such as bright sunlight, moderate light, low light, and even artificial light. This challenge may distort color, texture, and edge-based features.

Background Complexity
Plants in campus often surrounded by other vegetation, artificial objects, and human. They may act as noise and bias that disturb feature extraction.

Intra-Class Variability
Different samples in the same plant species may have totally different shape, size, and appearance, which can largely lower the performance of the model.

Limited Data and Imbalanced Conditions
There are the limited number of instances for each species in campus. Some of them have only a few instances, so that the same instance may appear across the training, validation, and test sets. This leads to instance-level data leakage, making the model to memorize individuals rather than learning general species traits.

1.2 Problem Formulation

The purpose of this project to classify an input image of a plant to one of the predefined plant classes. This task can be described as the below expression.

Input: Image $x \in \mathbb{R}^{H \times W \times 3}$

Output: Class label $y \in \{1,2,\ldots,K\}$

where K is the number of plant species considered in the dataset.

1.3 Approach Overview

To comply with the constraint of not using deep learning approaches, this project employs 1) **traditional handcrafted feature extraction** and 2) **classical machine learning models**. Figure 1 below shows the overall pipeline.

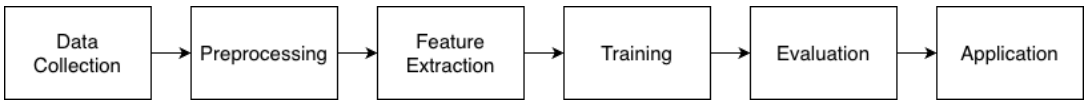

Figure 1: Overall System Pipeline: Data Collection → Preprocessing → Feature Extraction → Model Training → Evaluation → Application

The key components of the proposed pipeline are:

1. Handcrafted Feature Extraction
This program extracts four visual features from images:

- Histogram of Oriented Gradients (HOG) for shape and edge information
- Local Binary Patterns (LBP) for texture recognition
- HSV color histograms for color distribution
- Gray-Level Co-occurrence Matrix (GLCM) for additional texture statistics

These features are concatenated to form a fixed length of 1D feature vector for each image.

2. Classical Machine Learning Models
We evaluate multiple classifiers, including Support Vector Machines (SVM), Random Forest (RF), and Extreme Gradient Boosting (XGBoost), to determine the best performing model for this task.

3. Data Augmentation and Robustness Strategy
Data augmentation improves generalization by applying flipping, brightness adjustment, rotation, and cropping to the training set after data splitting. This order prevents information leakage between train, validation, and test sets.

4. Systematic Experimental Design
We found the best pipeline components with a series of controlled experiments:

- The effect of data augmentation

- Feature ablation to find the best feature combinations
- The influence of dataset refinement

2. Dataset Construction

The plant images for this dataset are taken manually in a university campus. They are captured in real-world conditions such as lighting, viewpoint, distance, and background. These variations help a traditional computer vision model remain robust under natural noises

2.1 Data Collection

8 plant species are selected for target classes for this dataset:

1. *Azalea*
2. *Banana Shrub*
3. *Beautiful Bougainvillea*
4. *Buddhist Pine*
5. *Camellia Japonica*
6. *Dragon Juniper*
7. *Golden Leaves*
8. *Indian Sublease Tree*

For each class, images were captured under different lighting, angle, and distance. The objective was to gather clear images of every plant while also capturing the diversity of actual campus ecosystems.

All images except for 4 photos are captured by me. The below four of *Banana Shrub* are shared by Mok Pan Yu (22112435d):

- DarkLight_9441.png
- DarkLight_9442.png
- DarkLight_9443.png
- DarkLight_9445.png

The final class list is stored in models/class_names.json to ensure consistency across source codes. This strategy prevents inclusion of unwanted classes.

In addition, only species with correct name plates are selected to avoid wrong labeling. These label images are surely not used for model training or evaluation to prevent data leakage.

Figure 2: Example Plant Images and Corresponding Name-Plate Images

2.2 Dataset Organization

The dataset consists of raw, train, validation, and test set. The raw folder basically has all images, and they are divided into train, val, and test folders, and images in the raw are distributed into 3 other folders.

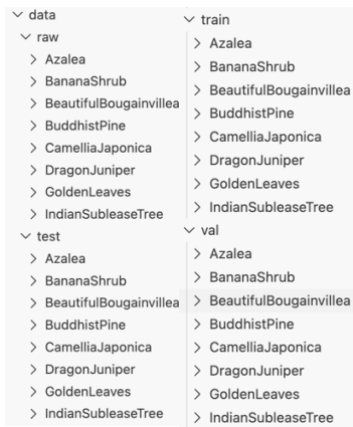

Figure 3: Overall Directory Structure

2.3 Split Strategy

The images in the raw folder are split into train, validation, and test in 60:20:20 ratio. A more technical strategy was adopted rather than fully random split.

To prevent uneven distribution and bias, each class in the dataset was first partitioned by lighting conditions – such as *BrightLight, MidLight, and DarkLight* – as specified in the name of each image. Assuming that consecutive images are nearly identical, a block-based partition, dividing sorted images into segments, was applied within these groups. This method ensures more generalization and avoidance of overestimation of model performance by deploying similar images in different subset.

2.4 Dataset-Specific Constraints

Despite efforts to collect diverse images, the dataset is subject to several practical constraints arising from real-world data collection.

First, some classes have inconsistent lighting coverage due to environmental reasons. For example, *Azalea* at night was not dark enough due to artificial light from the building, and *Dragon Juniper* in the morning was slightly shaded by covered walkaway compared to other classes in Bright Lighting.

Secondly, some classes have only a limited number of instances available on campus. In case of *Indian Sublease Tree*, only one instance was there in front of library, which leads to the model just memorizing the shape and color of the individual rather than learning generalized features.

Figure 4: *Azalea* under *DarkLight* and *Dragon Juniper* under *BrightLight*

3. Methodology

3.1 Overall Pipeline

The pipeline shown in Figure 1 follows 1) preprocessing, 2) feature extraction, 3) model training, and 4) evaluation.

3.2 Image Preprocessing

Each input image is resized to a resolution of $\mathbf{128 \times 128}$ **pixels** to make consistent feature dimensions.

Each image is converted into two representations:

- a **grayscale image** for gradient and texture-based features
- a **color image** for color-based feature extraction

This separated representation allows different feature descriptors to operate on the most appropriate input format.

3.3 Feature Representation

To capture the diverse visual characteristics of plants, a combination of complementary handcrafted features was employed. Instead of relying on a single descriptor, multiple features were used to represent shape, texture, and color information.

Design Motivation

To address the aformentioned challenges of real-world plant classification, a single feature is not sufficient to represent all patterns and detail of plant images. Therefore, we experiment 4 feature combinations for the optimal robustness and performance of the model

Features

Histogram of Oriented Gradients (HOG)

HOG detects shape of an object by calculating intensity gradients of pixels within an image. It is effective for object detection because it focuses on edges and local shapes. This program uses 9 orientations, a cell size of 8×8 pixels, and a block size of 2×2 cells with L2-Hys normalization

Local Binary Patterns (LBP)

LBP finds **texture** by comparing center's intensities to its neighborhood. This is expected to capture surface patterns such as leaf and stem textures. LBP with $P = 8$ and $R = 1$ is used and histogram is normalized as well.

Hue, Saturation, and Value (HSV) Color Histogram

Color information is represented using a 3D histogram in HSV color space with 8 bins per channel. RGB is considered relatively inaccurate since it is often easily influenced by light condition. For example, a green leaf under sunlight has (R=40 G=200 B=40), but when it comes under shadow, it becomes (R=20 G=100 B=20), which is quite farther than human intuition. Since our dataset contains various light conditions, HSV shows more robust performance than RGB.

Gray-Level Co-occurrence Matrix (GLCM)

GLCM also computes spatial relationships between pixel intensities to find **texture**. This program quantizes grayscale images into 8 partitions, and statistical properties are computed over multiple angles.

Feature Combination

The selected features are concatenated into a 1D feature vector to be inputted to the model.

The code for feature extraction pipeline is systematically designed to enable configuration for experiments of different set of features, such as HOG only, HOG + LBP, HOG + LBP + HSV, and others.

3.4 Model and Training

This multi-class classification is conducted by three machine learning models:

- **Support Vector Machine (SVM)** as a baseline model
- **Random Forest (RF)** as an ensemble-based model
- **Extreme Gradient Boosting (XGBoost)** as a strong non-linear model

For SVM, feature vectors are standardized using **StandardScaler** to ensure balanced feature scaling.

The extracted feature vectors serve as fixed-length inputs to the classifiers. Model training is performed on the training dataset, and performance is evaluated on a validation set using accuracy and F1-based metrics.

All experiments are conducted under controlled settings, where only one factor (e.g., feature set or augmentation) is varied at a time to ensure fair comparison.

3.5 Application Design

For Task 5, a simple application was developed to demonstrate the practical usability of the system by StreamLit library.

The application takes a single input image and outputs the predicted plant class. The input image is processed using the same pipeline as in training, including resizing, feature extraction, and feature concatenation. The resulting feature vector is then passed to the pre-trained model to make a prediction.

With reference to Figure 5, an image of Buddhist Pine from the internet is inputted and correctly classified. This task ensures that the system can be used in a real-world inference setting, rather than being limited to offline evaluation.

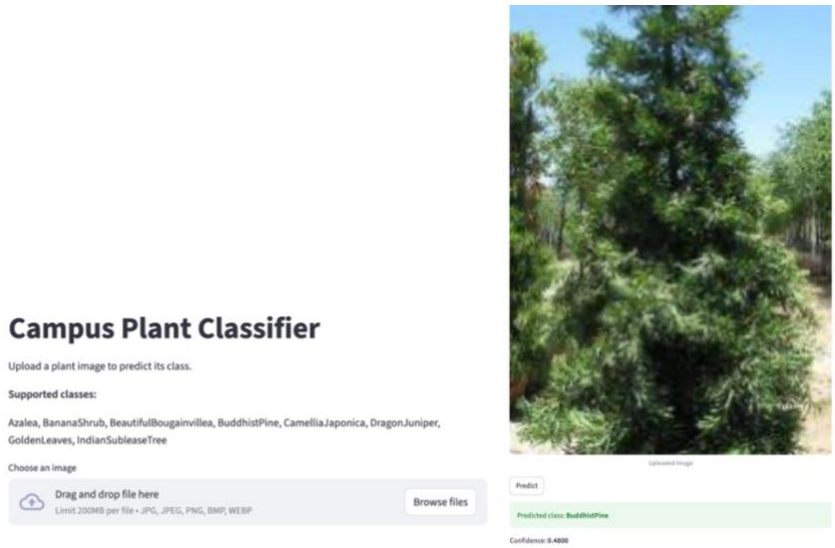

Figure 5: Web Application Page and the Prediction of Buddhist Pine

The steps to launch the application is introduced in ***README.md*** in codebase.

3.6 Codebase Structure

This assignment is implemented with modular structure, consisting of split_dataset.py, train.py, evaluate.py, app.py, and some others. Each model can be operated independently for flexibility and maintainability. For more detail, refer Appendix A.

4. Experiments

In this section, 4 experiments are conducted to find the most suitable model to classify plant images. As mentioned earlier, images from 8 classes are distributed to Train, Val, Test set in ratio of 6:2:2.

Class	Train	Val	Test	Total
Azalea	15	5	6	26
BananaShrub	10	4	6	20
BeautifulBougainvillea	14	4	7	25
BuddhistPine	14	4	7	25
CamelliaJaponica	13	4	6	23

DragonJuniper	13	4	7	24
GoldenLeaves	15	5	5	25
IndianSubleaseTree	11	3	5	19

Table 1: Dataset Distribution

Evaluation Metrics

Model performance is evaluated by **Accuracy**, **Macro F1 score**, and **Weighted F1 score**. Accuracy represents the overall model performance. While Weighted F1 score calculates the average F1 score of all classes with considering the number of images distributed to each class, Macro F1 score just computes the average F1 score of all classes evenly.

All experiments were conducted within this fixed dataset split. Only validation set was used for evaluating model comparison, whereas test set was for final test.

4.1 Baseline Experiment

All three models – Support Vector Machines (SVM), Random Forest (RF), and Extreme Gradient Boosting (XGB) - are evaluated under the same configuration:

- Feature set = HOG+LBP+HSV
- Feature dimension = 8100 (HOG) + 10 (LBP) + 512 (HSV) = 8622
- With the same augmentation to train set
- Image size = 128×128
- Fixed Data Split

Model	Accuracy	Macro F1	Weighted F1
SVM	0.545	0.496	0.499
Random Forest	**0.758**	**0.723**	**0.737**
XGBoost	0.697	0.690	0.686

Table 2: Baseline Models Comparison

Result Analysis

Firstly, SVM achieved the lowest performance in all metrics. SVM was generally considered as a good model under high dimensions and small sample size. Even with scaling, SVM struggled to draw proper complex decision boundaries under such high dimensional feature space.

RF significantly outperformed SVM, proving that ensemble-based tree models are more suitable to capture interaction among shape, color, and texture features. Additionally, XGB failed to perform better than RF, possibly due to the limited sample size.

4.2 Augmentation Experiment

Several augmentation techniques – horizontal flipping, brightness adjusting, small rotation, and slight cropping and zooming – are applied to supplement limited sample sizes, applied only to the training set to prevent data leakage. As shown in Table 3, Augmentation increased the size of training set by three times. Like the previous experiment, experiments were conducted under the same setting as the baseline.

Setting	Number of Images
Before Augmentation	105
After Augmentation	420

Table 3: The number of Training Images Before and After Augmentation

Model	Accuracy (No Aug)	Accuracy (Aug)	Macro F1 (No Aug)	Macro F1 (Aug)
SVM	0.545	0.576	0.496	0.541
Random Forest	0.758	**0.909**	0.723	**0.898**
XGBoost	0.697	0.848	0.690	0.841

Table 4: Effect of Data Augmentation

Result Analysis

According to the Table 4, SVM only showed marginal improvement in performance. However, RF and XGB substantially improved after augmentation. This suggests that increasing size of training set is effective for complex models to capture variations in plant images.

Observation

It is highly unusual to see this huge improvement in performance only by adopting augmentation. This topic will be handled in the next section.

4.3 Feature Ablation

Based on a comprehensive analysis of previous experiments, the RF model is selected as a final model. To optimize the model performance, several combinations of features were tested as shown in the Table 5.

Feature Set	**Accuracy**	**Macro F1**
HOG	0.333	0.308
HOG + LBP	0.394	0.319
HOG + HSV	0.909	0.898
HOG + LBP + HSV	0.909	0.898
HOG + LBP + HSV + GLCM	0.909	0.898
HSV only	**0.970**	**0.968**

Table 5: Feature Ablation Results

Result Analysis

Interestingly, the result clearly states that only color information is highly important for this task. Using solely HOG poorly performed, adding LBP marginally improved. The combination with HSV largely improved to higher than 0.90. This may suggest that texture and shape features do not clearly appear in this dataset.

Observation

One plausible reason that may explain this consequence is that many images, especially those captured from a distance or under dark-light conditions, do not clearly contain leaf or stem textures and shapes. Consequently, HOG, LBP and GLCM does not lead to better performance.

Moreover, higher feature dimensions did not lead to better performance. For instance, HOG alone produces more than 8000 dimensions, which does not improve the

performance. This suggest that higher dimensions do not mean more information, and feature quality and relevance were much more effective to predict the target class.

The most serious concern of this study is the potential presence of instance-level overlap among dataset, meaning that images of the identical plant instance are spread over Train, Val, and Test set, resulting in overly optimistic validation accuracy. This problem will be handled more deeply in Section 5.

4.4 Final Model Selection and Tuning

As mentioned in Section 4.3, **Random Forest with HSV and Augmentation** has been selected as a final model setting. The reason why feature ablation was not considered for model selection is that experimenting every combination of features for each model is time consuming.

We hypothesize that the inherent capabilities of RF, capturing non-linear relationships among features, generalization and robustness against noises, contributed to the superior performance in this task.

Hyperparameter Tuning

Subsequently, a grid search over core parameters of RF, *n_estimators, max_depth, and min_samples_split*, was conducted to optimize the model's validation accuracy. These parameters are briefly introduced as below:

- *n_estimators* = [100, 200, 300]: This parameter limits the number of decision trees in the ensemble. Increasing number of trees usually reduces variance and avoids overfitting.
- *max_depth* = [None, 10, 20]: This defines the maximum depth of each decision tree. A deeper tree can learn more complex pattern but is also susceptible to overfitting.
- *min_samples_split* = [2, 5]: This decides the threshold of the minimum number of samples required to remain in each leaf node of trees. Higher value generally leads to robustness against noises and overfitting.

As a result, the following set of parameters is selected with no improvement of validation accuracy: **[*n_estimators*: 100, *max_depth*: None, *min_samples_split*: 2]**

5. Error Analysis and Limitation

Despite the nearly perfect performance on validation accuracy of the model, the one on the test set dropped a lot. This suggests that the model was not generalized at all. Hence,

this section analyzes the potential reasons for model errors and structural limitations of the dataset and feature selection.

5.1 Gap between Validation and Test Performance

The final model, Random Forest with HSV and augmentation, performed exceptionally well on the validation set, whereas its performance on the test set was much lower.

One possible reason is poorly designed dataset, which has limited number of species and their instances on campus. The best design is to make sure that instances between sets do not overlap, preventing instance-level data leakage. Even though each image of the identical instance was captured in different angles and lightings, moderate amount of information is inevitably shared, and the model can learn the instance-specific features to reduce training error.

Furthermore, one potential point is about the deterministic block-based split within each lighting group. Since samples are indexed in the order they were captured, a block based split assigns samples from different days into separate sets. Therefore, it is possible that more challenging samples with unseen variations went into the test set by chance. This could have prevented an opportunity for the model to learn essential features, and relatively simple or clean images were distributed into training and validation sets.

For these reasons, the validation accuracy should be cautionally interpreted for this task. Following the standard practice of Data Science, the final model is still selected based on its validation accuracy. The issues discussed in the following sections are rooted from this problem as well.

5.2 Over-Reliance on Color Feature

According to the feature ablation experiment with combinations of HSV, HOG, LBP, and GLCM conducted on Section 4.3, only HSV was chosen for training the final model based on validation accuracy. Although this decision was methodologically reasonable, the model performed significantly worse on the test set than on the validation set.

The limitation of this decision is that it is highly vulnerable to lighting changes and may fail to distinguish classes with similar color histograms. Majority of plants is dominantly green, and they should be discriminated by other features, such as shapes and textures. HSV also cannot capture spatial distribution of colors.

To sum up, HSV was indeed the strongest feature, but not enough to generalize the model.

5.3 Class-Wise Error Patterns

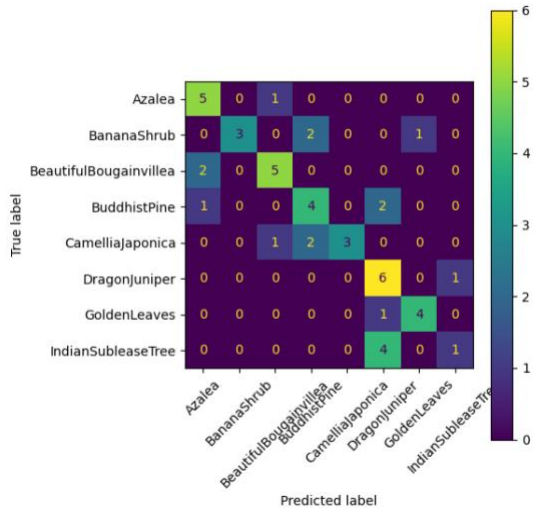

Figure 6: Confusion Matrix of the Final Model on the Test Set

The confusion matrix in Figure 6 identifies several misclassification patterns. While some classes like *Azalea, Beautiful Bougainvillea, and Dragon Juniper* were predicted quite accurately, the model struggled to distinguish others.

A major misclassified pattern is that *Indian Sublease Tree* was mostly predicted as *Dragon Juniper*. By investigating images with such pattern in results/misclassified/true_IndianSubleaseTree_pred_DragonJuniper, all four wrong samples were under mid and dark lights, and they share strong color similarities with *Dragon Junipers*.

Figure 7: Mutually Misclassified Images of *IndianSubleaseTree* and *DragonJuniper*

The second noteworthy pattern in confusion matrix is *Banana Shrub* and *Camellia Japonica* predicted as *Buddhist Pine*. This demonstrate that they also share similar color distribution.

Lastly, since both *Azalea and Beautiful Bougainvillea* have pink flowers, the model mutually confused between the two classes.

Figure 8: Mutually Misclassified Images of *Azalea* and *BeautifulBougainvillea*

These class-wise error patterns also directly shows that HSV-only model is not sufficient for classifying plant classes.

5.4 Solution

The aforementioned issues are rooted from the structure of the dataset. They can be only solved by rebuilding dataset. There must be no instance overlaps between sets for validation set to function properly, then experiments can find genuine features and configurations for the best model performance and generalization.

6. Use of Generative AI

In this task, the following two Generative AI (Gen AI) are used:

- ChatGPT Plus for complex reasoning
- Gemini 3 Flash for validating answers from ChatGPT

They support primarily for:

- Data Collection Strategy
- Choosing models to be experimented
- The whole code implementation and refactoring

6.1 Strengths of GenAI

As time goes, the performance of GenAI gets increasingly powerful. It can effectively generate syntactically correct code, refactor the code to be more maintainable, and debug

existing errors. The presence of GenAI eliminates the need for developers to memorize boilerplate python syntax, significantly reducing the time to write code.

The power of GenAI also comes from solving complicated problems with reasoning, but for learning purposes, I tried to identify and solve problems coming from the structure of the dataset and pipeline by myself.

6.2 Limitations of GenAI

Several fallbacks still exist despite its advancement.

Firstly, GenAI was not able to understand the entire context of the codebase and dataset structure. Because of limited tokens provided by GenAI, it only works with partial information, and it sometimes produces answers that do not match the overall system structure.

Another limitation of GenAI was in reasoning about model performance globally and generalized; while it was able to propose improvements based on validation data, it couldn't determine problems like validation / test mismatch, or bias in the data set, without being provided specific guidance. Therefore, only human analysts were able to interpret the data from an experimental view.

Finally, GenAI still had limited ability to understand visual information of the dataset. Even though it can describe images, they should be uploaded one by one for the best performance. For example, for GenAI to analyze misclassified images, I can hardly send all images at one time, and it cannot precisely detect critical reasons for the wrong prediction. It still requires human guidance.

6.3 Cross-Model Verification Strategy

To avoid hallucination, outputs generated by ChatGPT for sophisticated queries were often verified by Gemini. Even though Gemini is a free model with weaker reasoning and performance, it was enough to increase reliability.

This strategy provided robustness of the development process from hallucination.

7. Conclusion

This project built a complete plant classification pipeline using handcrafted features and classical machine learning models. The workflow included dataset construction, preprocessing, feature extraction, model comparison, tuning, and final evaluation.

The experiments showed that Random Forest with HSV-based features achieved the best validation performance. However, the final test result revealed that strong validation scores did not necessarily translate into strong generalization. The most important lesson from this project is that a model can only be as reliable as the dataset it is trained and evaluated on.

In this study, many of the major problems were ultimately rooted in dataset structure rather than model design alone. Limited plant instance diversity, possible overlap of similar instances across splits, uneven difficulty across classes, and lighting-related inconsistencies all affected the reliability of validation performance. As a result, feature selection, model comparison, and tuning were all influenced by the properties of the dataset.

Therefore, the project highlights the importance of systematic dataset construction in computer vision tasks. A carefully designed dataset with sufficient instance diversity, more balanced conditions, and stricter split control is essential for obtaining trustworthy evaluation results. Future work should focus not only on improving models, but also on improving the dataset itself, since better data is fundamental to better generalization

Reference

https://en.wikipedia.org/wiki/LLM-as-a-Judge

Appendix A: Code Structure

```
ASSIGNMENT2_CODE_21097305D_HEO_SUNGHAK/
│
├── data/
│   ├── train/
│   ├── val/
│   └── test/
│
├── models/
│   ├── experiments/                : store model
│   └── class_names.json
│
├── results/
│   ├── experiments/                : record the result and setting of each experiment
│   ├── misclassified/              : collect misclassified images
│   ├── sample_images/
│   ├── confusion_matrix.png
│   ├── experiment_history.jsonl
│   └── prediction_records.csv
```

```
│
├── src/
│   ├── app.py
│   ├── augmentation.py
│   ├── evaluate.py
│   ├── experiment_config.py
│   ├── experiment_utils.py
│   ├── feature_extraction.py
│   ├── sanity_check.py
│   ├── split_dataset.py
│   ├── train.py
│   └── tune.py
├── README.md
├── requirements.txt
└── dataset.txt
```